PROJECT DOCUMENTATION · VERSION 1.0



FEEL THE BEST QUALITY.

# ARSH PERFUME

## Website documentation

Customer guide · Administration · Architecture
Database · Installation · Operations

27 October 2026

Based on the source in this project workspace

# Contents

A practical guide for the store owner, administrators and developers. Read the customer/admin sections for daily use, the technical sections for implementation, and the appendices for exact schema and route details.


Project overview 03

Features and site map 04

Customer guide: discovery and cart 05

Customer guide: checkout and tracking 06

Customer accounts and Google sign-in 07

Admin guide: catalogue and inventory 08

Admin guide: orders and payments 09

Admin guide: content, customers and reports 10

Admin guide: store settings 11

Technology stack and structure 12

Architecture and request flow 13

Service responsibilities 14

Checkout integrity, pricing and inventory 15

Order lifecycle 16

Payment integrations 17

Database model and relationships 18

Security and access controls 19

Installation: local development 20

Environment and runtime configuration 21

Deployment, queues and maintenance 22

Validation and current limitations 23

Troubleshooting and handover 24

Database dictionary (all 36 tables) 25

Registered route reference (140 routes) 46

Glossary and document maintenance 54

# Project overview

ARSH PERFUME is an India-focused e-commerce website for attars, roll-on perfumes and fragrance oils. It combines a public storefront, customer accounts and an administrator back office in one Laravel application.

## Purpose and objectives

- Help customers discover fragrances, compare available sizes and place orders.
- Keep product prices, stock, discounts and order totals consistent on the server.
- Give the store owner tools to manage products, payments, fulfilment, content and reports.
- Support Cash on Delivery, Razorpay online payments and direct UPI with manual receipt verification.

## Users and access

| User | Main capabilities |
|---|---|
| Visitor / guest | Browse, search, manage a session cart, check out when enabled, track an order and contact the store. |
| Customer | Guest capabilities plus wishlist, saved addresses, profile, password and own order history. |
| Administrator | Restricted management screens for catalogue, inventory, orders, customers, content, settings and reports. |

## Scope of this document

This handbook explains the implemented website and how to install, use and maintain it. Appendices contain migration-derived table definitions and the registered application routes. It describes source behavior; it does not certify a production deployment or a real payment-provider integration.

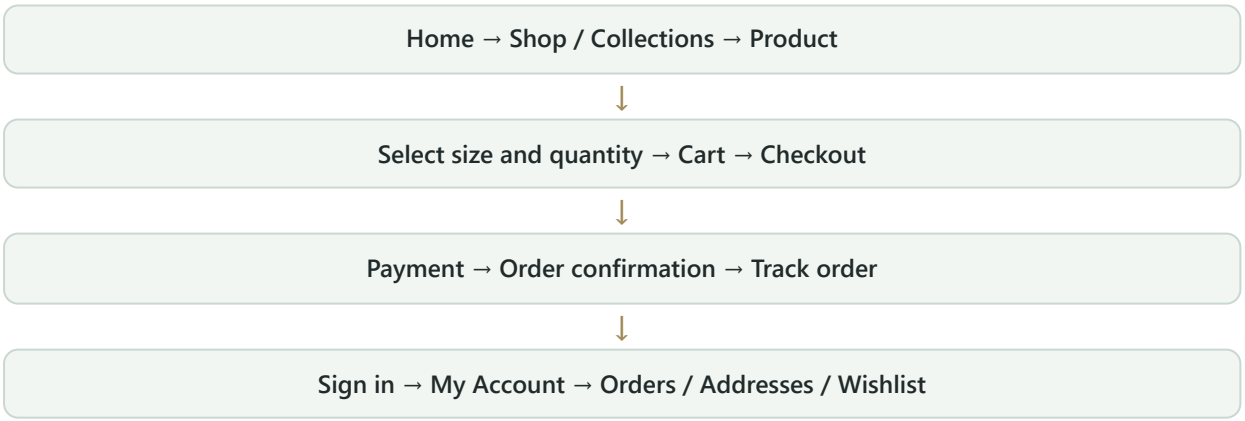
> **Evidence and date**
> Reviewed on 27 October 2026. The application route list was generated successfully. An isolated SQLite test run returned 68 passed and 2 failed with 389 assertions. The failures and their practical limits are recorded in the validation section.

`Source: composer.json; package.json; routes/; app/Services/`

# Features and site map

| Area | Implemented features |
|---|---|
| Discovery | Home, shop, collections, product detail, search, category/fragrance/price/stock filters and sorting. |
| Shopping | Variants, guest/customer cart, quantity changes, coupons and server-calculated totals. |
| Checkout | Guest or account checkout, Indian shipping address, COD, Razorpay and direct UPI QR. |
| Customer account | Email registration/login, password reset, optional Google sign-in, profile, addresses, own orders and cancellation. |
| Engagement | Wishlist, delivered-purchase reviews, FAQs, contact enquiries and newsletter subscription/unsubscribe. |
| Administration | Dashboard, products/images/sizes, categories, inventory, orders, invoices, customers, coupons and content. |
| Operations | Sales/inventory/customer reports, CSV exports, business settings, activity history, queued notifications and scheduled expiry. |
| Public content | About, contact, policy/help pages, sitemap.xml and robots.txt. |

## Main navigation paths




**Availability depends on configuration**

Draft products and unpublished policy pages are intentionally withheld. Guest checkout, COD, UPI and homepage sections depend on the store settings. Google sign-in requires server-side OAuth configuration.

Source: `routes/web.php; routes/auth.php; routes/admin.php`

# Customer guide: discovery and cart

## Browse and find a fragrance

1. Open `/shop` to view the catalogue, or `/collections` to browse categories.
2. Search by keyword. Narrow results by category, fragrance family, minimum/maximum price or in-stock availability.
3. Sort by newest, price ascending, price descending or popularity.
4. Open a product to read the supplied description, fragrance notes, specifications, images and approved reviews.

## Choose a size and add to cart

Prices and stock belong to product variants, such as a bottle size. Choose an active size with a confirmed price and available stock. The cart validates availability; each line allows up to 10 units and remains subject to current stock.

## Manage the cart

1. Open `/cart`. Adjust quantities or remove an item.
2. Enter a coupon if you have one. The server checks validity, eligible products, order minimums and usage limits.
3. Review the subtotal, discount, shipping, tax information and final amount.
4. Continue to checkout. Availability, stock and current prices are checked again when the order is placed.

## Wishlist and reviews

Sign in to save products to the wishlist. Reviews require a delivered purchase of the product and are held for administrator approval. A customer cannot submit multiple reviews of the same product.

> **Cart continuity**
> Guests use a browser-session cart. Signed-in users have a persistent cart. The application merges the guest cart during sign-in; the displayed quantities still depend on stock validation.

Source: app/Http/Controllers/ShopController.php; app/Services/CartService.php; app/Services/CouponService.php; app/Http/Controllers/ReviewController.php

# Customer guide: checkout and tracking

## Place an order

1. Review the cart and continue to `/checkout`. Sign in if guest checkout has been disabled by the store.
2. Provide your name, email, phone number and shipping address. The state must match a configured Indian state/territory and the PIN code must have six digits.
3. Select an available payment method. Review the final amount and submit once.
4. Complete the method-specific payment instructions. Retain the order number shown on confirmation.

| Method | Customer action | Payment confirmation |
|---|---|---|
| Cash on Delivery | Place the order and pay when collected. | Administrator collection or delivery updates payment records. |
| Razorpay | Complete the provider checkout. | Verified server callback/webhook; a redirect alone is insufficient. |
| Direct UPI QR | Scan/open the payment link and submit the transaction reference. | Administrator checks the actual bank receipt before marking paid. |

## Track and manage orders

Use `/track-order` with the order number and checkout email. Signed-in customers can also open My Account → Orders to view their own order details and history. Customers may cancel their own orders while pending or confirmed.

> **Guest order access**
>
> The browser session that placed a guest order receives access to its protected storefront pages. Keep the order number and email for the separate tracking lookup. A sequential internal database ID is not a customer access credential.

Source: `app/Http/Requests/CheckoutRequest.php`; `app/Http/Controllers/CheckoutController.php`; `app/Support/OrderAccess.php`; `app/Policies/OrderPolicy.php`

# Customer accounts and Google sign-in

## Email accounts

Customers register at `/register`, sign in at `/login` and sign out using the logout form. Forgotten-password and token-based reset screens are provided. Account screens support profile updates, password changes and saved-address management.

## Optional Google sign-in

- Laravel Socialite handles the Google OAuth redirect and callback.
- Google sign-in remains unavailable until a Client ID and Client Secret are configured.
- A new public Google registration creates a customer account, never an administrator.
- Verified Google email and a stable Google account identifier are required.
- An existing email match does not silently connect accounts. Existing customers sign in locally, then connect Google from My Account after confirming their password.
- Google-only customers can establish email/password sign-in through the account or password-reset flow.

## Configuration example

```
GOOGLE_CLIENT_ID=your-client-id.apps.googleusercontent.com
GOOGLE_CLIENT_SECRET=your-client-secret
GOOGLE_REDIRECT_URI=http://localhost:8000/auth/google/callback

php artisan config:clear
```

Register the exact callback URI with the OAuth provider. Scheme, hostname, port and path must match the address used to open the store. For production, use the actual HTTPS domain. If no redirect override is set, the application derives it from APP_URL.

> **Account access**
> Disabled accounts are blocked and signed out on their next web request. Customer order and address policies restrict records to the owning customer.

Source: app/Services/GoogleSignInService.php; routes/auth.php; docs/google-login-setup.md; app/Http/Middleware/EnsureUserIsActive.php

# Admin guide: catalogue and inventory

## Access the administration panel

Sign in using an active administrator account, then open `/admin`. There is no public administrator registration. Provision administrators with the dedicated command described in the installation section.

## Create or update a product

1. Open Products. Enter the name, SKU, category, description and optional fragrance information.
2. Add one or more variants. Each variant has its own SKU, size, price, sale price, stock and low-stock threshold.
3. Upload product images and select a primary image. Supported uploads: JPG/JPEG, PNG and WebP, up to 8 files; each at most 3 MB and at least 300 × 300 pixels.
4. Keep incomplete products as drafts. Publishing requires at least one active variant with a positive price. A sale price must be below the regular price.
5. Use featured/bestseller flags and supplied SEO fields where appropriate. Save and inspect the public product page.

## Categories and stock

Manage category names, slugs, descriptions, images, active status and sort order. A category that still contains products cannot simply be deleted; deactivate it or reassign the products first.

Use Inventory to adjust variant stock and record the reason. The inventory ledger records additions, deductions, reservations, cancellations and stocktake differences. Stock is reserved immediately during order placement, so available stock changes before fulfilment.

> **CSV support**
>
> Product CSV export is implemented. A product import route is registered, but its controller method is absent; CSV product import must not be described as a working feature.

Source: app/Http/Requests/ProductRequest.php; app/Http/Controllers/Admin/ProductController.php; app/Services/InventoryService.php

# Admin guide: orders and payments

## Process an order

1. Open Orders and review the customer's address, purchased item snapshots, totals and payment state.
2. Confirm payment where required. Unpaid online orders cannot advance through fulfilment.
3. Move the order through permitted states: confirm, process, ship, and deliver.
4. Add courier/tracking details as they become available. Review the recorded history and invoice.

## Payment actions

| Method | Administrator responsibility |
|---|---|
| COD | Record collected cash when appropriate; delivery automatically marks COD payment as collected. |
| Razorpay | Use verified gateway records. Investigate failed or mismatched payments before proceeding. |
| UPI | Inspect the receiving bank/UPI account, compare full amount and reference, confirm receipt, then use Verify UPI payment received. |

## Cancellation, returns and refunds

Cancellation is allowed only by the configured status transition map. Reserved stock is restored once on cancellation. Return restocking is optional. Refunds may be full or partial, up to the remaining refundable amount; a full refund changes the payment status to refunded.

Live Razorpay refunds use the gateway integration. For UPI, transfer the refund separately through the bank/UPI app before recording it in the store. A record-only action does not move money to a customer's bank account.

> **Direct UPI review**
> A customer-entered reference is a claim awaiting verification. The same verified bank reference cannot settle two orders. The Razorpay unpaid-order timeout excludes direct UPI orders; cancel stale UPI orders manually when appropriate.

Source: app/Http/Controllers/Admin/OrderController.php; app/Services/OrderService.php; app/Services/UpiPaymentService.php

# Admin guide: content, customers and reports

| Module | Typical tasks |
|---|---|
| Dashboard | Review order/revenue metrics, charts and recent operating information. |
| Customers | Inspect customer details and orders; activate/deactivate accounts. |
| Coupons | Configure fixed/percentage discounts, validity dates, eligible products/categories, minimum order and usage caps. |
| Banners | Maintain homepage promotional content, images and ordering. |
| Pages | Edit, preview, publish or unpublish informational and policy pages. |
| FAQs | Create and maintain customer help questions and answers. |
| Reviews | Approve, reject or delete submitted product reviews. |
| Enquiries | Read contact messages and maintain their handling state. |
| Newsletter | View subscriptions and export the subscriber list. |
| Reports | Review sales, inventory and customer reports; use supported exports. |
| Activity log | Inspect administrator actions and recorded changes. |

## Revenue definition

The shared revenue scope includes paid or refunded orders and excludes cancelled/returned orders. Net revenue per included order is grand total minus refunded amount. A pending COD order does not count as paid revenue until collection is recorded.

## Content publication

Seeded policy pages contain owner-confirmation placeholders and start unpublished. Replace and review those texts before publishing. Product claims, fragrance notes and legal/business details must come from the owner.

`Source: routes/admin.php; app/Models/Order.php; app/Services/ReportService.php; database/seeders/PageSeeder.php`

# Admin guide: store settings

Settings are stored in the database and exposed through Admin → Settings. They override defaults from `config/arsh.php`. Environment secrets remain in server configuration.

| Group | Examples and effect |
|---|---|
| Business | Store name, tagline, contact details, address, business hours, GSTIN, logo/favicon, about text, footer text and links. |
| Social | Instagram, Facebook, YouTube and WhatsApp links. |
| Commerce | Tax enabled/rate, inclusive pricing, flat shipping, free-shipping threshold, COD, guest checkout and low-stock settings. |
| UPI payments | Enable direct UPI and provide the payee name and UPI ID. All required receiving details must be valid. |
| Invoice | Invoice prefix and footer text. |
| Notifications | Administrative order-notification destination and notification settings. |
| Homepage | Hero text and visibility of catalogue, reviews, newsletter and social sections. |

## Setup sequence for the owner

1. Complete the business identity, contact details, branding and invoice text.
2. Confirm tax and shipping rules and decide whether to allow guest checkout and COD.
3. Configure the UPI receiving account only if direct UPI will be accepted.
4. Prepare products, variants, actual stock and product photography.
5. Review and publish content pages, then perform a controlled checkout rehearsal.

> **Receiving-account snapshots**
>
> Changing the UPI ID affects new checkouts. Existing UPI orders retain the receiving details captured when they were placed; verify against that order's recorded destination.

Source: `config/arsh.php; app/Models/Setting.php; app/Http/Controllers/Admin/SettingController.php`

# Technology stack and structure

| Layer | Project technology |
| --- | --- |
| Backend | PHP ^8.2; Laravel ^12.0; Eloquent ORM; custom authentication controllers. |
| Identity | Laravel Socialite ^5.31 for Google account sign-in. |
| Database | MySQL-oriented migrations; in-memory SQLite for the automated suite. |
| Rendering | Server-rendered Blade views with Tailwind CSS 4 and Alpine.js 3. |
| Assets | Vite 7 and laravel-vite-plugin 2; Node.js for building frontend assets. |
| Browser features | Chart.js 4 for admin charts, Framer Motion DOM APIs for motion, qrcode for local UPI QR generation, Axios. |
| Testing | PHPUnit ^11.5 with Laravel test helpers, factories and Mockery. |

## Directory guide

```
app/Http/Controllers/    Public, Auth and Admin request handlers
app/Http/Requests/       Checkout and product validation
app/Models/              Eloquent records and relationships
app/Services/            Business rules and operations
app/Policies/            Order and address authorization
app/Support/             Money, order access and helpers
bootstrap/app.php        Boot, routes and middleware
config/                  Laravel and store configuration
database/                Migrations, seeders and factories
resources/views/         Blade screens and components
resources/css|js/        Styles and browser behavior
routes/                  Web, auth, admin and scheduling
tests/                   Unit and feature tests
public/                  Web document root and built assets
storage/                 Runtime data, logs and uploads
```

The store is a single Laravel application with progressively enhanced server-rendered pages. Composer and npm lock files are present; keep them when installing reproducible dependencies.

Source: composer.json; package.json; vite.config.js

# Architecture and request flow




## Responsibilities

| Component | Role |
|---|---|
| Controllers | Coordinate the request and response; delegate business operations. |
| Services | Own reusable transaction, pricing, stock and payment rules. |
| Models | Represent tables, relationships, query scopes and record helpers. |
| Policies / middleware | Protect account ownership and administrative access. |
| Blade / JavaScript | Render content and add interactions, charts, QR generation and motion. |

## Background work

Order placement emits an event whose listener sends customer and administrative notifications. Queue workers deliver queued work. The scheduler expires unpaid Razorpay orders and prunes failed jobs. Webhooks enter through a signed provider endpoint rather than a customer session.

`Source: bootstrap/app.php; app/Events/OrderPlaced.php; app/Listeners/SendOrderPlacedNotifications.php; routes/console.php`

# Service responsibilities

| Service | Responsibility |
| --- | --- |
| CartService | Guest/customer carts, cart merging, item availability, quantities and summaries. |
| PricingService | Shared subtotal, discount, shipping and tax calculations in integer paise. |
| CouponService | Eligibility, validity periods, order minimums, usage limits and discount caps. |
| OrderService | Transactional placement, stock reservation, snapshots, state transitions, cancellation and refunds. |
| InventoryService | Controlled stock changes and inventory movement records. |
| RazorpayService | Provider API interactions, signed verification, simulated local gateway and refunds. |
| PaymentService | Verified payment handling, capture/failure records and payment state changes. |
| UpiPaymentService | Availability, account snapshots, payment URI, submitted references and admin receipt verification. |
| GoogleSignInService | Google identity validation, safe registration and explicit linking. |
| AdminProvisioner | Controlled creation/update of administrator accounts. |
| ReportService | Sales, inventory and customer report queries/aggregations. |
| ActivityLogger | Administrative audit entries. |

## Supporting helpers

- `Money`: converts decimal strings to/from integer paise and formats monetary amounts.
- `setting()`: reads a runtime setting with a fallback.
- `money()`: formats a money value for display.
- `media_url()`: builds a media URL or fallback.
- `OrderAccess`: centralizes storefront order access for owners or permitted browser sessions.

Source: app/Services/; app/Support/Money.php; app/Support/helpers.php; app/Support/OrderAccess.php

# Checkout integrity, pricing and inventory

## Transactional placement

1. Validate checkout fields and the selected method; obtain the cart and optional idempotency key.
2. Begin a database transaction and lock variants in stable ID order.
3. Recheck active product/variant status, confirmed prices and available stock.
4. Lock and revalidate a coupon where supplied. Calculate authoritative totals.
5. Create the order, item/address snapshots, payment record and status history.
6. Reserve stock immediately and write inventory movements; clear the cart.
7. Commit and emit the placed-order event. A repeated idempotency key returns the original order.

## Totals

Subtotal is the sum of current unit price × quantity. Discounts are bounded by eligible amounts. Shipping is evaluated against the discounted subtotal and the configured free-shipping threshold. With inclusive tax, the tax portion is informational; with exclusive tax, tax is added to the discounted subtotal. Shipping is then included in the final amount.

```
Discounted subtotal = subtotal - validated discount
Inclusive-tax total = discounted subtotal + shipping
Exclusive-tax total = discounted subtotal + tax + shipping
Net included revenue = grand total - refunded amount
```

## Preserved purchase history

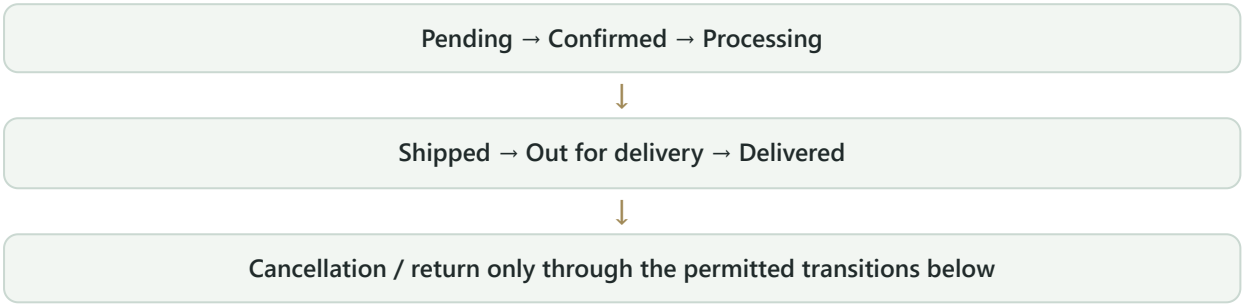
Order items snapshot product/variant information, quantities and prices. The delivery address is also copied to the order. Later edits to the catalogue or saved address do not rewrite an existing purchase.

> **Concurrency and stock**
>
> The server does not trust a browser-supplied total. Row locks and transactional stock operations protect checkout races. Cancellation uses stock_restored_at to avoid restoring the same reservation twice.

Source: app/Services/OrderService.php; app/Services/PricingService.php; app/Services/InventoryService.php

# Order lifecycle

Fulfilment status and payment status are separate fields. A shipment status change does not substitute for verified payment.




| Current status | Allowed next statuses |
|---|---|
| pending | confirmed, cancelled |
| confirmed | processing, cancelled |
| processing | shipped, cancelled |
| shipped | out_for_delivery, delivered, returned |
| out_for_delivery | delivered, returned |
| delivered | returned |
| cancelled | None (terminal) |
| returned | None (terminal) |

## Payment states and operating rules

- Order payment states: pending, paid, failed and refunded. Individual gateway payment records can use provider/attempt states such as created.
- Customers can cancel only pending or confirmed orders that they own.
- Unpaid online orders cannot advance through fulfilment.
- COD is marked paid on delivery or explicit cash collection.
- Unpaid Razorpay reservations expire after 30 minutes and are collected by the task running every ten minutes. Actual release occurs on the next successful scheduled run.
- Direct UPI awaiting review is excluded from that expiry task.

Source: config/arsh.php; app/Services/OrderService.php; app/Console/Commands/ExpireUnpaidOrders.php; routes/console.php

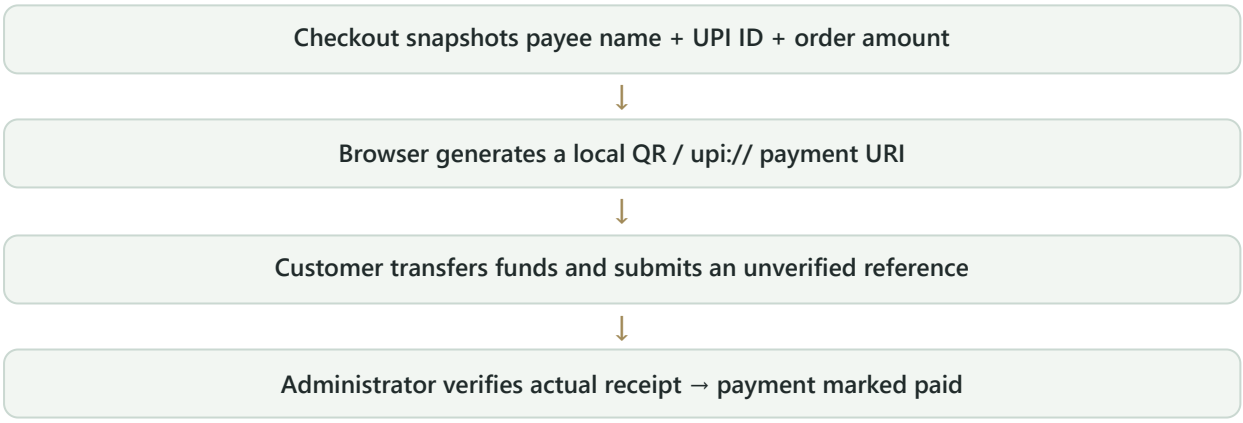
# Payment integrations

## Razorpay online payments

Configure RAZORPAY_KEY_ID, RAZORPAY_KEY_SECRET and RAZORPAY_WEBHOOK_SECRET on the server. Register `POST /webhooks/razorpay` as the provider webhook endpoint for capture and failure notifications. Use test credentials before configuring live credentials.

- The amount comes from the saved order total and is checked against provider values.
- Checkout callbacks use HMAC-SHA256 over order/payment identifiers; webhooks verify the raw request body.
- Recorded provider event identifiers make duplicate/retried webhooks idempotent.
- A successful browser redirect alone never marks an order paid.
- A simulated gateway exists for local non-production use. Production requires real provider configuration.

## Direct UPI QR




QR generation uses the bundled qrcode library. The code does not call an external QR-image service. References are normalized and must contain 6–64 letters or digits. A verified reference is unique across settled orders.

> **Financial verification boundary**
>
> This documentation run performed no real bank transfer, real Razorpay charge/refund or live Google consent round trip. Automated tests use controlled or simulated integration paths.

Source: app/Services/RazorpayService.php; app/Services/PaymentService.php; app/Services/UpiPaymentService.php; resources/js/

# Database model and relationships

The migration source declares **36 tables: 29 application tables and 7 framework support tables**. Laravel also creates the migrations bookkeeping table. The appendix records column definitions from migration source rather than private database contents.

| Domain | Tables |
|---|---|
| Accounts | roles, users, user_addresses |
| Catalogue | categories, products, product_images, product_variants, reviews |
| Shopping | carts, cart_items, wishlists, wishlist_items |
| Discounts | coupons, coupon_products, coupon_categories, coupon_usages |
| Orders | orders, order_items, order_status_histories |
| Payments | payments, payment_webhook_events |
| Inventory | inventory_movements |
| Content / operations | contact_enquiries, newsletter_subscribers, banners, pages, faqs, settings, admin_activity_logs |
| Framework | password_reset_tokens, sessions, cache, cache_locks, jobs, job_batches, failed_jobs |

## Core relationship map

```
Role 1 ── * User 1 ── * UserAddress
Category 1 ── * Product 1 ── * ProductVariant
                      ├── * ProductImage
                      └── * Review
Cart 1 ── * CartItem * ── 1 ProductVariant
User 1 ── * Order 1 ── * OrderItem
                ├── * Payment
                └── * OrderStatusHistory
ProductVariant 1 ── * InventoryMovement
Coupon ── products / categories / recorded usages
```

Money uses decimal persistence and integer-paise checkout arithmetic. Products are soft deleted. User Google identity/password-login fields were added in the October 2026 migration.

Source: database/migrations/; app/Models/; docs/DATABASE_REFERENCE.md

# Security and access controls

| Control | Implemented behavior |
| --- | --- |
| Authentication | Session-based login, hashed passwords, password reset, session refresh and optional Google OAuth. |
| Role protection | Admin routes require web, auth and admin middleware. Public forms do not assign administrator privileges. |
| Inactive accounts | Web middleware signs out disabled users and blocks continued access. |
| Record ownership | Policies and OrderAccess protect customer orders, addresses and permitted guest sessions. |
| CSRF | Web forms use CSRF protection. The Razorpay webhook exception relies on its HMAC signature. |
| Input checks | Form Requests and controller validation constrain checkout, products, images and other inputs. |
| Payment checks | Server totals, signed verification, amount checks, event deduplication and manual bank-receipt verification for UPI. |
| Upload handling | Raster image type/size/dimension checks and randomized storage names. |
| Auditability | Order histories, inventory movement records and administrator activity logs. |

## Configured rate limits

Per-minute limits: login 5; OAuth 10; registration 5; password reset 5; checkout 10; coupon attempts 15; tracking 10; contact 5; newsletter 5; reviews 5. Keys vary by endpoint and use IP, customer identity or email plus IP.

## Configuration and publication

Keep credentials in the server environment. Publish only the public document root, disable production debug output and use HTTPS. Customer data, passwords, real keys and private database records are deliberately excluded from this handbook.

Source: bootstrap/app.php; app/Providers/AppServiceProvider.php; app/Policies/; app/Http/Middleware/; app/Http/Requests/

# Installation: local development

## Prerequisites

PHP 8.2+ with the required Laravel/database extensions, Composer 2, Node.js compatible with the installed Vite version, npm and MySQL 8. Vite 7 requires Node 20.19+ or 22.12+ according to its installed package metadata. Enable pdo_mysql, mbstring, openssl, tokenizer, XML/DOM, ctype, fileinfo, curl and GD for the documented functionality; SQLite extensions are needed for the test suite.

## Fresh installation in PowerShell

```
Set-Location C:\xampp\htdocs\arsh-perfume
composer install
# Copy only when .env does not already exist:
Copy-Item .env.example .env
php artisan key:generate
# Create a database, then configure DB_* in .env.
php artisan migrate --seed
npm ci
npm run build
php artisan storage:link
php artisan arsh:create-admin owner@example.com --name="Store Owner"
php artisan serve
```

The administrator command prompts for a password. Set APP_URL to the address you actually use; the default development address is `http://localhost:8000`. For day-to-day asset development, run `npm run dev` in a separate terminal.

## XAMPP configuration

Point the Apache virtual host at `C:/xampp/htdocs/arsh-perfume/public`, enable mod_rewrite and AllowOverride All, and start MySQL. The PHP CLI and Apache PHP configurations may differ. Ensure the web user can write storage and bootstrap/cache. Windows storage-link creation may require Developer Mode or an appropriately privileged terminal.

> **Fresh setup only**
>
> Do not replace an existing .env, regenerate a deployed APP_KEY, reseed owner data or rebuild an existing database just to read this documentation. The handwritten SQL schema predates the Google migration; migrations are authoritative.

Source: .env.example; composer.json; node_modules/vite/package.json; database/seeders/

# Environment and runtime configuration

| Variables | Purpose |
|---|---|
| APP_NAME / APP_ENV / APP_DEBUG / APP_URL | Application identity, environment, diagnostics and canonical base URL. |
| APP_KEY | Application encryption key. Create once on a fresh setup and retain securely. |
| DB_CONNECTION / DB_HOST / DB_PORT / DB_DATABASE / DB_USERNAME / DB_PASSWORD | Database driver and connection. Use a dedicated production database account. |
| SESSION_DRIVER / SESSION_SECURE_COOKIE | Session storage and HTTPS-only cookie behavior. |
| CACHE_STORE / QUEUE_CONNECTION | Cache and background-job drivers. |
| MAIL_* | Mailer, SMTP endpoint, credentials and sender identity. The log driver does not deliver email. |
| MEDIA_DISK / FILESYSTEM_DISK / AWS_* | Upload/storage configuration. Cloud disks require compatible configured adapters. |
| RAZORPAY_* | Gateway API ID/secret and webhook verification secret. |
| GOOGLE_CLIENT_ID / GOOGLE_CLIENT_SECRET / GOOGLE_REDIRECT_URI | OAuth credentials and optional exact callback override. |
| ADMIN_NAME / ADMIN_EMAIL / ADMIN_PASSWORD | One-time seeder/provisioning inputs. Remove after initial use. |
| BCRYPT_ROUNDS | Password-hash cost. |

## Local database example — placeholders only

```
APP_ENV=local
APP_DEBUG=true
APP_URL=http://localhost:8000
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=arsh_perfume
DB_USERNAME=arsh_app
DB_PASSWORD=replace-with-your-local-password
```

After changing environment configuration, clear cached values with `php artisan config:clear`. Rebuild production configuration caches during deployment. Change owner-facing business options through Admin → Settings.

Source: .env.example; config/; docs/CONFIG_REFERENCE.md

# Deployment, queues and maintenance

## Production release sequence

1. Prepare PHP, MySQL, HTTPS and a web-server document root pointing to public/. Keep environment secrets outside version control.
2. Set APP_ENV=production, APP_DEBUG=false, the HTTPS APP_URL and SESSION_SECURE_COOKIE=true.
3. Install locked PHP dependencies and build frontend assets. Apply migrations using the normal release procedure and an available backup.
4. Verify the storage link and writable runtime directories. Cache configuration/routes/views/events.
5. Run the scheduler every minute and maintain a supervised queue worker. Restart workers after a release.
6. Verify owner content, shipping/tax settings, notifications and payment methods with controlled test orders.

```
composer install --no-dev --optimize-autoloader
npm ci
npm run build
php artisan migrate --force
php artisan storage:link
php artisan config:cache
php artisan route:cache
php artisan view:cache
php artisan event:cache
php artisan queue:restart
```

## Background processes

```
# Keep the worker running under a service manager:
php artisan queue:work --sleep=3 --tries=3 --max-time=3600
# Invoke once per minute using cron / Windows Task Scheduler:
php artisan schedule:run
```

The schedule expires unpaid Razorpay reservations every ten minutes and prunes failed jobs older than 168 hours daily. The built-in health route is `/up`. Back up the database, uploaded media and deployment configuration; keep protected copies and rehearse restoration. Review application logs and failed jobs regularly.

Source: routes/console.php; bootstrap/app.php; config/queue.php; config/logging.php

# Validation and current limitations

## Verification performed for this document

| Check | Observed result on 27 October 2026 |
|---|---|
| Registered routes | php artisan route:list --json completed; 140 registered routes, including framework/support routes. |
| Automated suite | php artisan test --compact using an explicit testing environment and in-memory SQLite: 68 passed, 2 failed; 389 assertions. |
| Source inspection | Routes, services, middleware, validators, migrations, seeders, manifests and project references reviewed. |
| Live integration | No real Razorpay charge/refund, bank payment, Google consent or production deployment verified in this documentation task. |

## Two existing test failures

- `AdminProductTest::valid image is stored with random name`: CLI PHP lacks the GD extension, so the test cannot generate its fixture image.
- `PaymentWebhookTest::capture with wrong amount is recorded but does not pay the order`: the assertion expects payment-attempt status pending, while the actual record remains created. The order is still pending; the disagreement is about the attempt state.

## Known source gaps

- CSV product import has a registered route but no controller import method.
- Category ordering uses numeric sort values; drag-and-drop ordering is not implemented.
- The opening README status and some historical notes predate the installed dependencies and later tests.
- Product seeds are incomplete drafts; policy seeds are unpublished owner-confirmation placeholders.
- Automatic SMS/WhatsApp notifications, multi-currency checkout, multilingual content and a dedicated external search engine are not implemented.

> **Interpreting test coverage**
> A partly passing suite is useful evidence, not a claim that all features are ready for production. Resolve the two failures and exercise real configured integrations before treating the installation as fully validated.

Source: `tests/; phpunit.xml; routes/admin.php; app/Http/Controllers/Admin/ProductController.php`

# Troubleshooting and handover

| Symptom | First checks |
| --- | --- |
| Blank/error page | Inspect storage/logs/laravel.log, runtime directory permissions, PHP extensions and environment configuration. |
| Unstyled pages / missing assets | Run npm ci and npm run build; verify public/build/manifest.json and stale Vite hot-file state. |
| Database connection error | Check MySQL availability and DB_* values, then clear configuration cache. Do not recreate existing tables as a first step. |
| Images missing | Verify the configured media disk, uploaded file and public storage link. |
| Google button disabled | Set server OAuth credentials, confirm exact callback URI and clear cached configuration. |
| Email not delivered | Check mail driver/settings, worker health and failed jobs. The log driver only writes a log. |
| Online order remains unpaid | Inspect verified provider records, webhook URL/signature/amount and delivery logs; for UPI, verify actual receipt manually. |
| Stock remains reserved | Check the scheduler and expiry task for unpaid Razorpay orders; review stale UPI orders manually. |
| Administrator access denied | Confirm an active admin account created through arsh:create-admin and check session state. |
| Image test fails | Enable GD in the PHP CLI configuration used to run the test command. |

## Owner handover checklist

- Administrator access and recovery are established.
- Products, sizes, prices, stock, images and claims are confirmed.
- Business contact details, shipping/tax rules and content pages are reviewed.
- Payment integrations and receipt/refund procedures are rehearsed.
- Mail, queue, scheduler, monitoring and restore procedures are assigned.

## Source references

Start with README.md, docs/ARCHITECTURE.md, docs/DEVELOPER_GUIDE.md, docs/CONFIG_REFERENCE.md and docs/google-login-setup.md. Prefer current code and migrations where historical notes disagree. The PDF's appendices are a source snapshot, not a database dump.

`Source: docs/; README.md; app/; database/migrations/`

# Database dictionary · 1

## roles

| Column / declaration | Definition |
|---|---|
| id | id() |
| name | string('name', 40)->unique() |
| label | string('label', 80) |
| created_at, updated_at | timestamps() |

## users

| Column / declaration | Definition |
|---|---|
| id | id() |
| name | string('name') |
| email | string('email')->unique() |
| phone | string('phone', 20)->nullable() |
| email_verified_at | timestamp('email_verified_at')->nullable() |
| password | string('password') |
| role_id | foreignId('role_id')->constrained('roles')->restrictOnDelete() |
| is_active | boolean('is_active')->default(true) |
| last_login_at | timestamp('last_login_at')->nullable() |
| remember_token | rememberToken() |
| created_at, updated_at | timestamps() |
| google_id | string('google_id')->nullable()->unique() |
| password_login_enabled | boolean('password_login_enabled')->default(true) |
| Indexes / constraints | index(['role_id', 'is_active']). |

# Database dictionary · 2

## password_reset_tokens

| Column / declaration | Definition |
|---|---|
| email | string('email')->primary() |
| token | string('token') |
| created_at | timestamp('created_at')->nullable() |

## sessions

| Column / declaration | Definition |
|---|---|
| id | string('id')->primary() |
| user_id | foreignId('user_id')->nullable()->index() |
| ip_address | string('ip_address', 45)->nullable() |
| user_agent | text('user_agent')->nullable() |
| payload | longText('payload') |
| last_activity | integer('last_activity')->index() |

## cache

| Column / declaration | Definition |
|---|---|
| key | string('key')->primary() |
| value | mediumText('value') |
| expiration | integer('expiration') |

## cache_locks

| Column / declaration | Definition |
|---|---|
| key | string('key')->primary() |
| owner | string('owner') |
| expiration | integer('expiration') |

# Database dictionary · 3

## jobs

| Column / declaration | Definition |
|---|---|
| id | id() |
| queue | string('queue')->index() |
| payload | longText('payload') |
| attempts | unsignedTinyInteger('attempts') |
| reserved_at | unsignedInteger('reserved_at')->nullable() |
| available_at | unsignedInteger('available_at') |
| created_at | unsignedInteger('created_at') |

## job_batches

| Column / declaration | Definition |
|---|---|
| id | string('id')->primary() |
| name | string('name') |
| total_jobs | integer('total_jobs') |
| pending_jobs | integer('pending_jobs') |
| failed_jobs | integer('failed_jobs') |
| failed_job_ids | longText('failed_job_ids') |
| options | mediumText('options')->nullable() |
| cancelled_at | integer('cancelled_at')->nullable() |
| created_at | integer('created_at') |
| finished_at | integer('finished_at')->nullable() |

# Database dictionary · 4

## failed_jobs

| Column / declaration | Definition |
|---|---|
| id | id() |
| uuid | string('uuid')->unique() |
| connection | text('connection') |
| queue | text('queue') |
| payload | longText('payload') |
| exception | longText('exception') |
| failed_at | timestamp('failed_at')->useCurrent() |

## user_addresses

| Column / declaration | Definition |
|---|---|
| id | id() |
| user_id | foreignId('user_id')->constrained()->cascadeOnDelete() |
| label | string('label', 40)->nullable() |
| name | string('name') |
| phone | string('phone', 20) |
| address_line1 | string('address_line1') |
| address_line2 | string('address_line2')->nullable() |
| city | string('city', 100) |
| state | string('state', 100) |
| postal_code | string('postal_code', 10) |
| country | string('country', 60)->default('India') |
| is_default | boolean('is_default')->default(false) |
| created_at, updated_at | timestamps() |
| Indexes / constraints | index(['user_id', 'is_default']). |

# Database dictionary · 5

## categories

| Column / declaration | Definition |
|---|---|
| id | id() |
| name | string('name') |
| slug | string('slug')->unique() |
| description | text('description')->nullable() |
| image_path | string('image_path')->nullable() |
| sort_order | unsignedInteger('sort_order')->default(0) |
| is_active | boolean('is_active')->default(true) |
| created_at, updated_at | timestamps() |
| Indexes / constraints | index(['is_active', 'sort_order']). |

# Database dictionary · 6

## products

| Column / declaration | Definition |
|---|---|
| id | id() |
| category_id | foreignId('category_id')->nullable()->constrained('categories')->restrictOnDelete() |
| sku | string('sku', 64)->unique() |
| name | string('name') |
| slug | string('slug')->unique() |
| short_description | string('short_description', 300)->nullable() |
| description | longText('description')->nullable() |
| fragrance_family | string('fragrance_family', 60)->nullable()->index() |
| top_notes | text('top_notes')->nullable() |
| heart_notes | text('heart_notes')->nullable() |
| base_notes | text('base_notes')->nullable() |
| longevity | string('longevity', 120)->nullable() |
| projection | string('projection', 120)->nullable() |
| specifications | json('specifications')->nullable() |
| is_featured | boolean('is_featured')->default(false) |
| is_bestseller | boolean('is_bestseller')->default(false) |
| status | string('status', 16)->default('draft') |
| meta_title | string('meta_title')->nullable() |
| meta_description | string('meta_description', 320)->nullable() |
| owner_notes | text('owner_notes')->nullable() |
| sales_count | unsignedInteger('sales_count')->default(0) |
| deleted_at | softDeletes() |
| created_at, updated_at | timestamps() |
| Indexes / constraints | index(['status', 'is_featured']); index(['status', 'is_bestseller']); index(['status', 'created_at']); index('name'); index('sales_count'). |

# Database dictionary · 7

## product_images

| Column / declaration | Definition |
|---|---|
| id | id() |
| product_id | foreignId('product_id')->constrained()->cascadeOnDelete() |
| path | string('path') |
| alt | string('alt')->nullable() |
| sort_order | unsignedInteger('sort_order')->default(0) |
| is_primary | boolean('is_primary')->default(false) |
| created_at, updated_at | timestamps() |
| Indexes / constraints | index(['product_id', 'sort_order']). |

## product_variants

| Column / declaration | Definition |
|---|---|
| id | id() |
| product_id | foreignId('product_id')->constrained()->cascadeOnDelete() |
| name | string('name', 80) |
| size_ml | unsignedSmallInteger('size_ml')->nullable() |
| sku | string('sku', 64)->unique() |
| price | decimal('price', 10, 2)->nullable() |
| sale_price | decimal('sale_price', 10, 2)->nullable() |
| stock | unsignedInteger('stock')->default(0) |
| low_stock_threshold | unsignedInteger('low_stock_threshold')->default(5) |
| is_active | boolean('is_active')->default(true) |
| sort_order | unsignedInteger('sort_order')->default(0) |
| created_at, updated_at | timestamps() |
| Indexes / constraints | index(['product_id', 'is_active']); index('stock'). |

# Database dictionary · 8

## carts

| Column / declaration | Definition |
|---|---|
| id | id() |
| user_id | foreignId('user_id')->nullable()->unique()->constrained()->cascadeOnDelete() |
| session_token | string('session_token', 64)->nullable()->unique() |
| coupon_code | string('coupon_code', 50)->nullable() |
| created_at, updated_at | timestamps() |

## cart_items

| Column / declaration | Definition |
|---|---|
| id | id() |
| cart_id | foreignId('cart_id')->constrained()->cascadeOnDelete() |
| product_variant_id | foreignId('product_variant_id')->constrained('product_variants')->cascadeOnDelete() |
| quantity | unsignedSmallInteger('quantity')->default(1) |
| created_at, updated_at | timestamps() |
| Indexes / constraints | unique(['cart_id', 'product_variant_id']). |

## wishlists

| Column / declaration | Definition |
|---|---|
| id | id() |
| user_id | foreignId('user_id')->unique()->constrained()->cascadeOnDelete() |
| created_at, updated_at | timestamps() |

# Database dictionary · 9

## wishlist_items

| Column / declaration | Definition |
|---|---|
| id | id() |
| wishlist_id | foreignId('wishlist_id')->constrained()->cascadeOnDelete() |
| product_id | foreignId('product_id')->constrained()->cascadeOnDelete() |
| created_at, updated_at | timestamps() |
| Indexes / constraints | unique(['wishlist_id', 'product_id']). |

## coupons

| Column / declaration | Definition |
|---|---|
| id | id() |
| code | string('code', 50)->unique() |
| description | string('description')->nullable() |
| type | string('type', 12) |
| value | decimal('value', 10, 2) |
| min_order_amount | decimal('min_order_amount', 10, 2)->nullable() |
| max_discount | decimal('max_discount', 10, 2)->nullable() |
| starts_at | timestamp('starts_at')->nullable() |
| expires_at | timestamp('expires_at')->nullable() |
| usage_limit | unsignedInteger('usage_limit')->nullable() |
| per_user_limit | unsignedInteger('per_user_limit')->nullable() |
| used_count | unsignedInteger('used_count')->default(0) |
| is_active | boolean('is_active')->default(true) |
| created_at, updated_at | timestamps() |
| Indexes / constraints | index(['is_active', 'expires_at']). |

# Database dictionary · 10

## coupon_products

| Column / declaration | Definition |
|---|---|
| coupon_id | foreignId('coupon_id')->constrained()->cascadeOnDelete() |
| product_id | foreignId('product_id')->constrained()->cascadeOnDelete() |
| Indexes / constraints | primary(['coupon_id', 'product_id']). |

## coupon_categories

| Column / declaration | Definition |
|---|---|
| coupon_id | foreignId('coupon_id')->constrained()->cascadeOnDelete() |
| category_id | foreignId('category_id')->constrained()->cascadeOnDelete() |
| Indexes / constraints | primary(['coupon_id', 'category_id']). |

# Database dictionary · 11

## orders

| Column / declaration | Definition |
|---|---|
| id | id() |
| order_number | string('order_number', 32)->unique() |
| user_id | foreignId('user_id')->nullable()->constrained()->restrictOnDelete() |
| idempotency_key | string('idempotency_key', 64)->nullable()->unique() |
| status | string('status', 24)->default('pending') |
| payment_method | string('payment_method', 24) |
| payment_status | string('payment_status', 24)->default('pending') |
| name | string('name') |
| email | string('email') |
| phone | string('phone', 20) |
| address_line1 | string('address_line1') |
| address_line2 | string('address_line2')->nullable() |
| city | string('city', 100) |
| state | string('state', 100) |
| postal_code | string('postal_code', 10) |
| country | string('country', 60)->default('India') |
| notes | text('notes')->nullable() |
| subtotal | decimal('subtotal', 12, 2) |
| discount_total | decimal('discount_total', 12, 2)->default(0) |
| shipping_total | decimal('shipping_total', 12, 2)->default(0) |
| tax_total | decimal('tax_total', 12, 2)->default(0) |
| grand_total | decimal('grand_total', 12, 2) |
| refunded_amount | decimal('refunded_amount', 12, 2)->default(0) |
| coupon_id | foreignId('coupon_id')->nullable()->constrained('coupons')->nullOnDelete() |

# Database dictionary · 12

## orders (continued)

| Column / declaration | Definition |
|---|---|
| coupon_code | string('coupon_code', 50)->nullable() |
| courier_name | string('courier_name', 100)->nullable() |
| tracking_number | string('tracking_number', 100)->nullable() |
| tracking_url | string('tracking_url')->nullable() |
| placed_at | timestamp('placed_at')->nullable() |
| paid_at | timestamp('paid_at')->nullable() |
| shipped_at | timestamp('shipped_at')->nullable() |
| delivered_at | timestamp('delivered_at')->nullable() |
| cancelled_at | timestamp('cancelled_at')->nullable() |
| cancel_reason | string('cancel_reason')->nullable() |
| stock_restored_at | timestamp('stock_restored_at')->nullable() |
| ip_address | string('ip_address', 45)->nullable() |
| created_at, updated_at | timestamps() |
| Indexes / constraints | index(['user_id', 'status']); index('status'); index('payment_status'); index('payment_method'); index('email'); index('created_at'). |

# Database dictionary · 13

## order_items

| Column / declaration | Definition |
|---|---|
| id | id() |
| order_id | foreignId('order_id')->constrained()->cascadeOnDelete() |
| product_id | foreignId('product_id')->nullable()->constrained('products')->restrictOnDelete() |
| product_variant_id | foreignId('product_variant_id')->nullable()->constrained('product_variants')->restrictOnDelete() |
| product_name | string('product_name') |
| variant_name | string('variant_name', 80)->nullable() |
| sku | string('sku', 64) |
| description_snapshot | text('description_snapshot')->nullable() |
| image_path | string('image_path')->nullable() |
| list_price | decimal('list_price', 10, 2) |
| unit_price | decimal('unit_price', 10, 2) |
| quantity | unsignedSmallInteger('quantity') |
| line_total | decimal('line_total', 12, 2) |
| created_at, updated_at | timestamps() |
| Indexes / constraints | index('order_id'); index('product_id'). |

# Database dictionary · 14

## payments

| Column / declaration | Definition |
|---|---|
| id | id() |
| order_id | foreignId('order_id')->constrained()->restrictOnDelete() |
| provider | string('provider', 24) |
| provider_order_id | string('provider_order_id', 80)->nullable()->index() |
| provider_payment_id | string('provider_payment_id', 80)->nullable()->unique() |
| amount | decimal('amount', 12, 2) |
| currency | string('currency', 3)->default('INR') |
| status | string('status', 24)->default('created') |
| method | string('method', 40)->nullable() |
| error_code | string('error_code', 80)->nullable() |
| error_description | string('error_description')->nullable() |
| verified_at | timestamp('verified_at')->nullable() |
| meta | json('meta')->nullable() |
| created_at, updated_at | timestamps() |
| Indexes / constraints | index(['order_id', 'status']). |

## payment_webhook_events

| Column / declaration | Definition |
|---|---|
| id | id() |
| provider | string('provider', 24) |
| event_id | string('event_id', 100) |
| event_type | string('event_type', 80) |
| payload | json('payload')->nullable() |
| processed_at | timestamp('processed_at')->nullable() |
| outcome | string('outcome')->nullable() |
| created_at, updated_at | timestamps() |
| Indexes / constraints | unique(['provider', 'event_id']). |

# Database dictionary · 15

## coupon_usages

| Column / declaration | Definition |
|---|---|
| id | id() |
| coupon_id | foreignId('coupon_id')->constrained()->restrictOnDelete() |
| order_id | foreignId('order_id')->constrained()->cascadeOnDelete() |
| user_id | foreignId('user_id')->nullable()->constrained()->nullOnDelete() |
| email | string('email') |
| discount_amount | decimal('discount_amount', 10, 2) |
| created_at, updated_at | timestamps() |
| Indexes / constraints | unique(['coupon_id', 'order_id']); index(['coupon_id', 'user_id']); index(['coupon_id', 'email']). |

## order_status_histories

| Column / declaration | Definition |
|---|---|
| id | id() |
| order_id | foreignId('order_id')->constrained()->cascadeOnDelete() |
| from_status | string('from_status', 24)->nullable() |
| to_status | string('to_status', 24) |
| note | string('note')->nullable() |
| changed_by | foreignId('changed_by')->nullable()->constrained('users')->nullOnDelete() |
| created_at, updated_at | timestamps() |
| Indexes / constraints | index(['order_id', 'created_at']). |

# Database dictionary · 16

## inventory_movements

| Column / declaration | Definition |
|---|---|
| id | id() |
| product_variant_id | foreignId('product_variant_id')->constrained('product_variants')->cascadeOnDelete() |
| type | string('type', 30) |
| quantity_change | integer('quantity_change') |
| stock_after | unsignedInteger('stock_after') |
| reference_type | string('reference_type', 40)->nullable() |
| reference_id | unsignedBigInteger('reference_id')->nullable() |
| user_id | foreignId('user_id')->nullable()->constrained()->nullOnDelete() |
| note | string('note')->nullable() |
| created_at | timestamp('created_at')->useCurrent() |
| Indexes / constraints | index(['product_variant_id', 'created_at']); index(['reference_type', 'reference_id']); index('type'). |

# Database dictionary · 17

## reviews

| Column / declaration | Definition |
|---|---|
| id | id() |
| product_id | foreignId('product_id')->constrained()->cascadeOnDelete() |
| user_id | foreignId('user_id')->constrained()->restrictOnDelete() |
| order_id | foreignId('order_id')->nullable()->constrained('orders')->nullOnDelete() |
| rating | unsignedTinyInteger('rating') |
| title | string('title')->nullable() |
| body | text('body') |
| status | string('status', 16)->default('pending') |
| moderated_by | foreignId('moderated_by')->nullable()->constrained('users')->nullOnDelete() |
| moderated_at | timestamp('moderated_at')->nullable() |
| created_at, updated_at | timestamps() |
| Indexes / constraints | unique(['product_id', 'user_id']); index(['product_id', 'status']); index('status'). |

# Database dictionary · 18

## contact_enquiries

| Column / declaration | Definition |
|---|---|
| id | id() |
| name | string('name') |
| email | string('email') |
| phone | string('phone', 20)->nullable() |
| subject | string('subject')->nullable() |
| message | text('message') |
| status | string('status', 20)->default('new') |
| admin_notes | text('admin_notes')->nullable() |
| ip_address | string('ip_address', 45)->nullable() |
| handled_by | foreignId('handled_by')->nullable()->constrained('users')->nullOnDelete() |
| created_at, updated_at | timestamps() |
| Indexes / constraints | index('status'); index('created_at'). |

## newsletter_subscribers

| Column / declaration | Definition |
|---|---|
| id | id() |
| email | string('email')->unique() |
| consent_given_at | timestamp('consent_given_at') |
| consent_ip | string('consent_ip', 45)->nullable() |
| unsubscribe_token | string('unsubscribe_token', 64)->unique() |
| unsubscribed_at | timestamp('unsubscribed_at')->nullable() |
| created_at, updated_at | timestamps() |

# Database dictionary · 19

## banners

| Column / declaration | Definition |
|---|---|
| id | id() |
| title | string('title') |
| subtitle | string('subtitle')->nullable() |
| image_path | string('image_path')->nullable() |
| button_text | string('button_text', 60)->nullable() |
| link_url | string('link_url')->nullable() |
| position | string('position', 20)->default('promo') |
| sort_order | unsignedInteger('sort_order')->default(0) |
| is_active | boolean('is_active')->default(true) |
| starts_at | timestamp('starts_at')->nullable() |
| ends_at | timestamp('ends_at')->nullable() |
| created_at, updated_at | timestamps() |
| Indexes / constraints | index(['position', 'is_active', 'sort_order']). |

## pages

| Column / declaration | Definition |
|---|---|
| id | id() |
| slug | string('slug')->unique() |
| title | string('title') |
| content | longText('content')->nullable() |
| meta_title | string('meta_title')->nullable() |
| meta_description | string('meta_description', 320)->nullable() |
| is_published | boolean('is_published')->default(false) |
| reviewed_at | timestamp('reviewed_at')->nullable() |
| reviewed_by | foreignId('reviewed_by')->nullable()->constrained('users')->nullOnDelete() |
| created_at, updated_at | timestamps() |

# Database dictionary · 20

## faqs

| Column / declaration | Definition |
|---|---|
| id | id() |
| question | string('question') |
| answer | text('answer') |
| sort_order | unsignedInteger('sort_order')->default(0) |
| is_active | boolean('is_active')->default(true) |
| created_at, updated_at | timestamps() |
| Indexes / constraints | index(['is_active', 'sort_order']). |

## settings

| Column / declaration | Definition |
|---|---|
| id | id() |
| key | string('key')->unique() |
| value | text('value')->nullable() |
| group | string('group', 40)->default('general') |
| type | string('type', 16)->default('string') |
| created_at, updated_at | timestamps() |
| Indexes / constraints | index('group'). |

# Database dictionary · 21

## admin_activity_logs

| Column / declaration | Definition |
|---|---|
| id | id() |
| user_id | foreignId('user_id')->nullable()->constrained()->nullOnDelete() |
| action | string('action', 60) |
| subject_type | string('subject_type', 60)->nullable() |
| subject_id | unsignedBigInteger('subject_id')->nullable() |
| description | string('description') |
| properties | json('properties')->nullable() |
| ip_address | string('ip_address', 45)->nullable() |
| user_agent | string('user_agent')->nullable() |
| created_at | timestamp('created_at')->useCurrent() |
| Indexes / constraints | index('action'); index(['subject_type', 'subject_id']); index('created_at'). |

# Route reference · 1

GET routes also accept HEAD. Middleware names are shown as registered; admin routes use the web/auth/admin stack. These are browser/session endpoints, not a separate public JSON API. The product-import route is incomplete as noted earlier.

| Method | Path / route name | Middleware |
|---|---|---|
| GET | `/`<br>`home` | web |
| GET | `/about`<br>`pages.about` | web |
| GET | `/account`<br>`account` | web, auth |
| POST | `/account/addresses`<br>`account.addresses.store` | web, auth |
| PUT | `/account/addresses/{address}`<br>`account.addresses.update` | web, auth |
| DELETE | `/account/addresses/{address}`<br>`account.addresses.destroy` | web, auth |
| POST | `/account/google/connect`<br>`account.google.connect` | web, auth, throttle:oauth |
| GET | `/account/orders`<br>`account.orders.index` | web, auth |
| GET | `/account/orders/{order}`<br>`account.orders.show` | web, auth |
| POST | `/account/orders/{order}/cancel`<br>`account.orders.cancel` | web, auth |
| PUT | `/account/password`<br>`account.password.update` | web, auth |
| PATCH | `/account/profile`<br>`account.profile.update` | web, auth |
| GET | `/admin`<br>`admin.dashboard` | web, auth, admin |
| GET | `/admin/activity-log`<br>`admin.activity-log.index` | web, auth, admin |
| GET | `/admin/banners`<br>`admin.banners.index` | web, auth, admin |
| POST | `/admin/banners`<br>`admin.banners.store` | web, auth, admin |
| GET | `/admin/banners/create`<br>`admin.banners.create` | web, auth, admin |
| PUT\|PATCH | `/admin/banners/{banner}`<br>`admin.banners.update` | web, auth, admin |

# Route reference · 2

| Method | Path / route name | Middleware |
|---|---|---|
| DELETE | `/admin/banners/{banner}`<br>`admin.banners.destroy` | web, auth, admin |
| GET | `/admin/banners/{banner}/edit`<br>`admin.banners.edit` | web, auth, admin |
| GET | `/admin/categories`<br>`admin.categories.index` | web, auth, admin |
| POST | `/admin/categories`<br>`admin.categories.store` | web, auth, admin |
| GET | `/admin/categories/create`<br>`admin.categories.create` | web, auth, admin |
| POST | `/admin/categories/reorder`<br>`admin.categories.reorder` | web, auth, admin |
| PUT\|PATCH | `/admin/categories/{category}`<br>`admin.categories.update` | web, auth, admin |
| DELETE | `/admin/categories/{category}`<br>`admin.categories.destroy` | web, auth, admin |
| GET | `/admin/categories/{category}/edit`<br>`admin.categories.edit` | web, auth, admin |
| PATCH | `/admin/categories/{category}/toggle`<br>`admin.categories.toggle` | web, auth, admin |
| GET | `/admin/coupons`<br>`admin.coupons.index` | web, auth, admin |
| POST | `/admin/coupons`<br>`admin.coupons.store` | web, auth, admin |
| GET | `/admin/coupons/create`<br>`admin.coupons.create` | web, auth, admin |
| PUT\|PATCH | `/admin/coupons/{coupon}`<br>`admin.coupons.update` | web, auth, admin |
| DELETE | `/admin/coupons/{coupon}`<br>`admin.coupons.destroy` | web, auth, admin |
| GET | `/admin/coupons/{coupon}/edit`<br>`admin.coupons.edit` | web, auth, admin |
| GET | `/admin/customers`<br>`admin.customers.index` | web, auth, admin |
| GET | `/admin/customers/{customer}`<br>`admin.customers.show` | web, auth, admin |

# Route reference · 3

| Method | Path / route name | Middleware |
|---|---|---|
| PATCH | `/admin/customers/{customer}/toggle`<br>`admin.customers.toggle` | web, auth, admin |
| GET | `/admin/enquiries`<br>`admin.enquiries.index` | web, auth, admin |
| GET | `/admin/enquiries/{enquiry}`<br>`admin.enquiries.show` | web, auth, admin |
| PATCH | `/admin/enquiries/{enquiry}`<br>`admin.enquiries.update` | web, auth, admin |
| GET | `/admin/faqs`<br>`admin.faqs.index` | web, auth, admin |
| POST | `/admin/faqs`<br>`admin.faqs.store` | web, auth, admin |
| GET | `/admin/faqs/create`<br>`admin.faqs.create` | web, auth, admin |
| PUT\|PATCH | `/admin/faqs/{faq}`<br>`admin.faqs.update` | web, auth, admin |
| DELETE | `/admin/faqs/{faq}`<br>`admin.faqs.destroy` | web, auth, admin |
| GET | `/admin/faqs/{faq}/edit`<br>`admin.faqs.edit` | web, auth, admin |
| GET | `/admin/inventory`<br>`admin.inventory.index` | web, auth, admin |
| GET | `/admin/inventory/movements`<br>`admin.inventory.movements` | web, auth, admin |
| POST | `/admin/inventory/{variant}/adjust`<br>`admin.inventory.adjust` | web, auth, admin |
| GET | `/admin/newsletter`<br>`admin.newsletter.index` | web, auth, admin |
| GET | `/admin/newsletter/export`<br>`admin.newsletter.export` | web, auth, admin |
| GET | `/admin/orders`<br>`admin.orders.index` | web, auth, admin |
| GET | `/admin/orders/export`<br>`admin.orders.export` | web, auth, admin |
| GET | `/admin/orders/{order}`<br>`admin.orders.show` | web, auth, admin |

# Route reference · 4

| Method | Path / route name | Middleware |
|---|---|---|
| GET | `/admin/orders/{order}/invoice`<br>`admin.orders.invoice` | web, auth, admin |
| POST | `/admin/orders/{order}/mark-paid`<br>`admin.orders.mark-paid` | web, auth, admin |
| POST | `/admin/orders/{order}/refund`<br>`admin.orders.refund` | web, auth, admin |
| PATCH | `/admin/orders/{order}/status`<br>`admin.orders.status` | web, auth, admin |
| PATCH | `/admin/orders/{order}/tracking`<br>`admin.orders.tracking` | web, auth, admin |
| POST | `/admin/orders/{order}/verify-upi`<br>`admin.orders.verify-upi` | web, auth, admin |
| GET | `/admin/pages`<br>`admin.pages.index` | web, auth, admin |
| PUT | `/admin/pages/{page}`<br>`admin.pages.update` | web, auth, admin |
| GET | `/admin/pages/{page}/edit`<br>`admin.pages.edit` | web, auth, admin |
| GET | `/admin/pages/{page}/preview`<br>`admin.pages.preview` | web, auth, admin |
| POST | `/admin/pages/{page}/publish`<br>`admin.pages.publish` | web, auth, admin |
| POST | `/admin/pages/{page}/unpublish`<br>`admin.pages.unpublish` | web, auth, admin |
| GET | `/admin/products`<br>`admin.products.index` | web, auth, admin |
| POST | `/admin/products`<br>`admin.products.store` | web, auth, admin |
| GET | `/admin/products/create`<br>`admin.products.create` | web, auth, admin |
| GET | `/admin/products/export`<br>`admin.products.export` | web, auth, admin |
| POST | `/admin/products/import`<br>`admin.products.import` | web, auth, admin |
| GET | `/admin/products/{product}`<br>`admin.products.show` | web, auth, admin |

# Route reference · 5

| Method | Path / route name | Middleware |
|---|---|---|
| PUT\|PATCH | `/admin/products/{product}`<br>`admin.products.update` | web, auth, admin |
| DELETE | `/admin/products/{product}`<br>`admin.products.destroy` | web, auth, admin |
| GET | `/admin/products/{product}/edit`<br>`admin.products.edit` | web, auth, admin |
| DELETE | `/admin/products/{product}/images/{image}`<br>`admin.products.images.destroy` | web, auth, admin |
| PATCH | `/admin/products/{product}/images/{image}/primary`<br>`admin.products.images.primary` | web, auth, admin |
| GET | `/admin/reports`<br>`admin.reports.index` | web, auth, admin |
| GET | `/admin/reports/customers`<br>`admin.reports.customers` | web, auth, admin |
| GET | `/admin/reports/inventory`<br>`admin.reports.inventory` | web, auth, admin |
| GET | `/admin/reports/sales`<br>`admin.reports.sales` | web, auth, admin |
| GET | `/admin/reviews`<br>`admin.reviews.index` | web, auth, admin |
| DELETE | `/admin/reviews/{review}`<br>`admin.reviews.destroy` | web, auth, admin |
| PATCH | `/admin/reviews/{review}/approve`<br>`admin.reviews.approve` | web, auth, admin |
| PATCH | `/admin/reviews/{review}/reject`<br>`admin.reviews.reject` | web, auth, admin |
| GET | `/admin/settings`<br>`admin.settings.edit` | web, auth, admin |
| PUT | `/admin/settings`<br>`admin.settings.update` | web, auth, admin |
| GET | `/auth/google/callback`<br>`auth.google.callback` | web, throttle:oauth |
| GET | `/auth/google/redirect`<br>`auth.google` | web, guest, throttle:oauth |
| GET | `/cancellation-policy`<br>`pages.cancellation` | web |

# Route reference · 6

| Method | Path / route name | Middleware |
|---|---|---|
| GET | `/cart`<br>`cart.index` | web |
| POST | `/cart/buy-now`<br>`cart.buy-now` | web |
| POST | `/cart/coupon`<br>`cart.coupon.apply` | web, throttle:coupon |
| DELETE | `/cart/coupon`<br>`cart.coupon.remove` | web |
| POST | `/cart/items`<br>`cart.items.store` | web |
| PATCH | `/cart/items/{item}`<br>`cart.items.update` | web |
| DELETE | `/cart/items/{item}`<br>`cart.items.destroy` | web |
| GET | `/checkout`<br>`checkout.show` | web |
| POST | `/checkout`<br>`checkout.store` | web, throttle:checkout |
| GET | `/checkout/pay/{order}`<br>`checkout.pay` | web |
| POST | `/checkout/pay/{order}/failed`<br>`checkout.failed` | web, throttle:checkout |
| POST | `/checkout/pay/{order}/simulate`<br>`checkout.simulate` | web, throttle:checkout |
| POST | `/checkout/pay/{order}/verify`<br>`checkout.verify` | web, throttle:checkout |
| GET | `/checkout/upi/{order}`<br>`checkout.upi` | web |
| POST | `/checkout/upi/{order}/reference`<br>`checkout.upi.reference` | web, throttle:checkout |
| GET | `/collections`<br>`collections.index` | web |
| GET | `/collections/{category}`<br>`collections.show` | web |
| GET | `/contact`<br>`contact` | web |

# Route reference · 7

| Method | Path / route name | Middleware |
|---|---|---|
| POST | `/contact`<br>`contact.store` | web, throttle:contact |
| GET | `/faq`<br>`pages.faq` | web |
| GET | `/forgot-password`<br>`password.request` | web, guest |
| POST | `/forgot-password`<br>`password.email` | web, guest, throttle:password-reset |
| GET | `/login`<br>`login` | web, guest |
| POST | `/login`<br>`(unnamed)` | web, guest, throttle:login |
| POST | `/logout`<br>`logout` | web, auth |
| POST | `/newsletter`<br>`newsletter.store` | web, throttle:newsletter |
| GET | `/newsletter/unsubscribe/{token}`<br>`newsletter.unsubscribe` | web |
| GET | `/order-success`<br>`order.success` | web |
| GET | `/payment-information`<br>`pages.payment` | web |
| GET | `/privacy-policy`<br>`pages.privacy` | web |
| GET | `/product/{product}`<br>`product.show` | web |
| POST | `/product/{product}/reviews`<br>`reviews.store` | web, auth, throttle:reviews |
| GET | `/register`<br>`register` | web, guest |
| POST | `/register`<br>`(unnamed)` | web, guest, throttle:register |
| POST | `/reset-password`<br>`password.store` | web, guest, throttle:password-reset |
| GET | `/reset-password/{token}`<br>`password.reset` | web, guest |

# Route reference · 8

| Method | Path / route name | Middleware |
|---|---|---|
| GET | /returns-refunds<br>pages.returns | web |
| GET | /robots.txt<br>robots | web |
| GET | /shipping-policy<br>pages.shipping | web |
| GET | /shop<br>shop | web |
| GET | /sitemap.xml<br>sitemap | web |
| GET | /storage/{path}<br>storage.local | — |
| PUT | /storage/{path}<br>storage.local.upload | — |
| GET | /terms<br>pages.terms | web |
| GET | /track-order<br>track-order | web |
| POST | /track-order<br>track-order.lookup | web, throttle:track |
| GET | /up<br>(unnamed) | — |
| POST | /webhooks/razorpay<br>webhooks.razorpay | web |
| GET | /wishlist<br>wishlist.index | web, auth |
| POST | /wishlist/{product}<br>wishlist.toggle | web, auth |

# Glossary and document maintenance

| Term | Meaning in this project |
|---|---|
| Variant | A separately priced and stocked size/option of a product. |
| Paise | Integer monetary unit used for checkout arithmetic; 100 paise = INR 1. |
| Snapshot | Purchase-time copy of item, address or UPI receiving details. |
| Reservation | Stock deduction when an order is placed, before shipment. |
| Idempotency | Repeated submissions/events produce one business effect. |
| Webhook | Signed server-to-server payment notification. |
| UTR / reference | Customer-submitted bank transaction reference; requires receipt verification. |
| Migration | Versioned table/schema change; authoritative database structure. |
| Seeder | Code creating initial roles, draft catalogue and content placeholders. |
| Queue worker | Background process delivering queued work such as notifications. |
| Scheduler | Periodic command runner used for expiry and maintenance. |
| CMS | Administration tools for content, banners, FAQs and published pages. |

## Regenerate this handbook

```
python docs/tools/build-project-pdf.py
```

The generator uses Python with Playwright and an installed Chromium browser. It reads migration reference text and runs the application's route-list command. It does not export environment secrets or query private store records. Narrative content and the dated test result are maintained in the generator; update them after implementation changes or a new verification run.

Outputs: `docs/ARSH_PERFUME_DOCUMENTATION.html` and `docs/ARSH_PERFUME_DOCUMENTATION.pdf`. The HTML embeds its brand image and styles for convenient editing and printing. PDF navigation includes linked contents and document outlines.

Source: docs/tools/build-project-pdf.py; docs/DATABASE_REFERENCE.md; routes/